CYBERSECURITY TRACK • PYTHON MODULE

Assignment 8

Password Generator & SHA-256 Hasher Tool

Done by: Abdulhadi Al-Ahdal

1. Project Overview

The objective of this assignment is to write a Python script that provides two security services: first, generating high-complexity random passwords using approved cryptographic libraries, and second, calculating cryptographic hash values using SHA-256 for text strings to illustrate secure password protection concepts[cite: 4].

2. Development Steps

1 **Password Function Creation**

Create a password generation function incorporating uppercase letters, lowercase letters, numbers, and special symbols[cite: 4].

2 **Secure Random Selection**

Utilize a cryptographically secure random library to pick characters based on the requested length[cite: 4].

3

4

Hash Function Building

Create a hash computation function that accepts text and generates its corresponding hash value[cite: 4].

Output Display

Display the generated password alongside its parallel cryptographic hash value in hexadecimal format[cite: 4].

3. Technical Features & Utilities

Secure Secrets Module: Use the cryptographically approved `secrets` library instead of traditional random packages to avoid prediction attacks[cite: 4].

Character Pooling: Aggregate character sets via `string.ascii_letters + string.digits + string.punctuation` [cite: 4].

4. Implementation Source Code

`password_hasher.py`

Python 3.x

```
import secrets
import string
import hashlib
import sys

def generate_secure_password(length=16):
    alphabet = string.ascii_letters + string.digits + string.punctuation
    # Use secrets module for cryptographically secure random choice
    password = ''.join(secrets.choice(alphabet) for _ in range(length))
```

```
    return password

def compute_sha256_hash(text):
    # Encode text and compute SHA-256 cryptographic hash
    return hashlib.sha256(text.encode('utf-8')).hexdigest()

def main():
    length = int(sys.argv[1]) if len(sys.argv) > 1 else 16

    print("=" * 75)
    print(" Python Security Tool: Password Generator & SHA-256 Hasher")
    print("=" * 75)

    generated_pwd = generate_secure_password(length)
    pwd_hash = compute_sha256_hash(generated_pwd)

    print(f" [+] Generated Password : {generated_pwd}")
    print(f" [+] SHA-256 Hash (Hex) : {pwd_hash}")
    print("=" * 75)

if __name__ == "__main__":
    main()
```

5. Acceptance Criteria

- **Security Compliant Password**
  Generation of a password matching security requirements with the specified length[cite: 4].
- **Accurate Hash Computation**
  100% accurate and correct SHA-256 calculation and extraction[cite: 4].